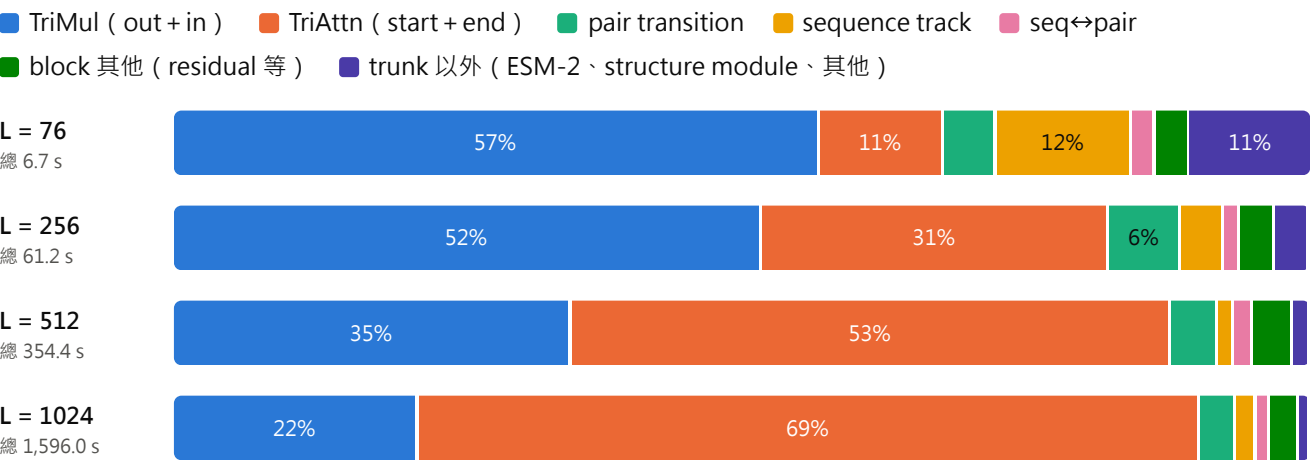


ESMFold 時間分布與結論 (workload characterization)

1. 整條流程的時間佔比隨 L 的變化



一條橫條 = 一次完整推論 (從胺基酸序列到 3D 結構) 的總時間，切成互不重疊的 7 組；長度依序為 76 (泛素) 、 256 、 512 、 1024 。各組包含的細項與秒數見表 1 。

- 1. 三角運算 (藍 + 橘) 隨 L 越佔越多，L = 1024 達 90% 。
- 2. TriMul (藍) 越來越小、TriAttn (橘) 越來越大，在 L = 256 ~ 512 之間交叉。
- 3. ESM-2 與 structure module (紫) 在 L ≥ 256 時幾乎看不到。

表 1 全部類別的時間與佔比 (192 次 block 呼叫的總和)

類別	L = 76		L = 256		L = 512		L = 1024	
	秒	%	秒	%	秒	%	秒	%
Triangle Multiplication (outgoing)	1.92	28.7	16.36	26.7	62.12	17.5	171.89	10.8
Triangle Multiplication (incoming)	1.89	28.2	15.31	25.0	61.95	17.5	172.11	10.8
Triangle Attention (starting node)	0.37	5.5	9.38	15.3	93.14	26.3	487.88	30.6
Triangle Attention (ending node)	0.36	5.4	9.33	15.2	93.82	26.5	609.17	38.2
pair transition (mlp_pair)	0.31	4.6	3.87	6.3	14.72	4.2	51.05	3.2
sequence attention	0.40	6.0	1.53	2.5	3.75	1.1	18.69	1.2
sequence transition (mlp_seq)	0.39	5.8	0.78	1.3	1.22	0.3	10.12	0.6
LayerNorm (sequence)	0.01	0.1	0.02	0.0	0.04	0.0	0.05	0.0
sequence → pair	0.11	1.6	0.64	1.0	4.20	1.2	11.85	0.7
pair → sequence	0.03	0.4	0.22	0.4	1.73	0.5	6.32	0.4
block 其他 (residual 加法等)	0.20	3.0	1.88	3.1	12.43	3.5	41.07	2.6
ESM-2 (語言模型)	0.52	7.8	1.18	1.9	2.88	0.8	6.40	0.4
structure module	0.19	2.8	0.61	1.0	2.06	0.6	8.03	0.5
其他 (embedding 、 recycling 等)	0.02	0.3	0.07	0.1	0.39	0.1	1.36	0.1
總時間	6.7	100	61.2	100	354.4	100	1,596.0	100
峰值記憶體 (整個程式)	19.9 GB		22.2 GB		27.4 GB		60.7 GB	

2. 結論

結論 1 整條流程的瓶頸是 pair 表示上的三角運算

- $L \geq 256$ 時，TriMul + TriAttn 佔整條流程 82 ~ 90% ; 48 個 block 合計 97 ~ 99%。
- ESM-2 + structure module : $L \geq 256$ 時不到 3%。ESM-2 佔全部參數的 80%，卻只佔不到 1% 的時間：參數量決定權重佔多少記憶體，時間則由中間資料 (pair 表示 L^2 、三角運算 L^3) 決定。
- 與 LightNobel 只加速 trunk、不加速 ESM-2 與 structure module 的選擇一致。

結論 2 瓶頸隨長度轉移：短序列 TriMul 最大，長序列 TriAttn 最大

- TriMul 佔比 57% $\rightarrow$ 52% $\rightarrow$ 35% $\rightarrow$ 22% ; TriAttn 11% $\rightarrow$ 31% $\rightarrow$ 53% $\rightarrow$ 69% ($L = 76 \rightarrow 1024$)。
- 兩者在 $L = 256 \sim 512$ 之間交叉。
- 原因：兩者都有 L^3 運算，但 TriMul 的資料只有 L^2 (每筆資料被重複使用 L 次)，TriAttn 會產生 L^3 大小的 logits ($L = 1024$ 時 17 GB)，搬運量跟著 L^3 成長。

結論 3 TriAttn：受記憶體頻寬限制，已貼近天花板

現象

- tri_att_start ($L = 1024$) : add_ 31% + softmax 28% = 59% 在讀寫 logits ; 矩陣乘法只佔 27%。
- add_ 的名目頻寬約 112 GB/s (計數的 68.7 GB $\div$ 實測 0.616 s)，機器上限約 100 GB/s (同種運算、同樣的記憶體擺放實測) $\rightarrow$ 已用滿頻寬。roofline 達成率 58 ~ 88%。
- $L = 1024$ 峰值記憶體 60.7 GB，主要是不分塊的 logits (一份 17 GB)。
- tri_att_end 在 $L = 1024$ 時比 start 慢 583 ms，其中 578 ms (99%) 來自 add_ (1,194 vs 616 ms) ; $L = 512$ 時兩者相同。

原因

- softmax 的依賴只需要「一列」 ($L = 1024$ 時 4 KB)，17 GB 的搬運來自「一個運算處理完整個張量」的執行方式。

結論 4 TriMul：時間大半浪費在資料排列，不在計算；只修好排列，整體可省約 10 ~ 13% (L = 1024)、19 ~ 24% (L = 512)

現象

- copy_ (permute 後的複製) 佔 TriMul 的 **50 ~ 58%**；einsum 本身只佔 14 ~ 17%。
- roofline 達成率只有 16 ~ 24%，三種運算中最低。
- permute 複製只有 9 GB/s，連續複製 76 ~ 82 GB/s，慢約 9 倍。
- **對照實驗**：同形狀 (128 個 1024×1024×1024)、資料已排好的矩陣乘法只要 **63 ms** (等於機器上限)；HF 的寫法是 einsum 149 ms + copy 436 ms = **585 ms**，差 **9.3 倍**。形狀本身沒有問題，損失全在排列。

原因

- bmm 要求 [c][i][k]，資料原本是 [i][k][c]。a、b 各轉一次、結果轉回一次，共 3 次。
- 同一 channel 的相鄰元素相隔 512 bytes，每抓一條 64 bytes 的 cache line 只用到 4 bytes。

效益估算 (只修好排列)

- 一次 tri_mul_out (L = 1024，881 ms) = A 組 (einsum + copy) 585 ms + B 組 (linear、gate、LayerNorm) 296 ms。
- 保守情境 (省掉進入前的 2 次 copy，結果仍要轉回 1 次)：A 組 63 + 124 = 187 ms → TriMul 快 1.82 倍 → 整體省 **9.7%**。
- 理想情境 (3 次 copy 全省)：A 組 63 ms → TriMul 快 2.45 倍 → 整體省 **12.8%**。
- L = 512：理想 einsum 未實測，以效率範圍估算，整體省 19 ~ 24%。
- 假設 B 組不變；若再把 B 組融合 (中間結果不寫回)，還能更省 → 以上為保守估計。只優化 TriMul 的上限：L = 512 最多 1.54 倍、L = 1024 最多 1.28 倍。